Airport Ground-Crew Coordination: A Multi-Agent Routing System

Name: Riya Solanki
Roll No.: BL.EN.U4CSE23065

1 Problem Statement

Airport ground operations require service vehicles to travel between a depot and aircraft gates to perform tasks such as fuelling, baggage handling, catering, and maintenance. This case study models four service-agent types operating in a grid-based airport environment. Each agent is assigned a destination, travels to the gate, performs its service, and returns to the depot when the scenario permits.

The system compares Cooperative A* and Greedy Best-First Search (GBFS). It must plan feasible routes, coordinate agents using the implemented reservation and conflict checks, and respond to scenario events such as gate unavailability and emergency priority. It also tests an unreachable destination. The objective is to evaluate route quality and search effort using distance, waiting time, makespan, detected conflicts, expanded nodes, planning runtime, and task-completion outcomes.

2 PEAS Description

PEAS stands for Performance measure, Environment, Actuators, and Sensors.

Component	Description
Performance measure	Reduce unnecessary travel and waiting; complete tasks; maintain a low makespan; avoid conflicts detected by the implemented checks; and measure nodes expanded and planning runtime.
Environment	A discrete airport grid containing walkable cells, obstacles, a depot, gates, four service-agent types, time-indexed reservations, and scenario events.
Actuators	Move to an adjacent cell, wait, follow a route, perform service, return to the depot, request route planning, and replan when required.
Sensors / inputs	Current position, assigned destination, agent status, gate availability, simulation time, scenario events, coordinator messages, and other agents' relevant reservations.

3 Environment Properties

- **Fully observable:** The planner has access to the modeled grid and relevant simulation state, including positions, destinations, gate status, and reservations.
- **Deterministic:** Given the same state, action, and predefined scenario rules, the modeled transition is predictable. A predefined gate closure makes the environment dynamic, not stochastic.
- **Sequential:** An action affects later states. An agent's movement and reservation can affect the routes available to other agents.
- **Dynamic:** Conditions can change during execution, for example when Gate B becomes unavailable or FUEL01 receives emergency priority.

- **Discrete:** Positions are represented by grid cells and planning proceeds in discrete time steps.
- **Multi-agent and cooperative:** Several agents operate in the same environment, and planning accounts for the reservations of agents planned earlier.
- **Known:** The map, movement rules, and scenario-event rules are supplied by the simulation.

4 Agent Model and Decision Cycle

The system models fuel, baggage, catering, and maintenance agents. An agent follows a perceive–decide–act cycle:

1. **Perceive:** Collect its position, destination, status, gate availability, simulation time, and relevant coordinator messages.
2. **Check messages:** Identify requests such as waiting, replanning, or notifications about an unavailable gate.
3. **Evaluate conditions:** Determine whether it is travelling, servicing, returning, finished, or needs a new route.
4. **Select a high-level action:** Follow a valid route, wait, service, return, or request route planning/replanning.
5. **Execute and repeat:** The simulation advances the agent and updates the state for the next time step.

The agent decision logic selects a high-level action; the search algorithm calculates the route; the coordinator manages priorities and reservations; and the simulation engine executes movements and scenario events. The emergency event is scenario-defined rather than independently detected by an agent.

5 Search Algorithms

5.1 Cooperative A*

A* evaluates a search state using

$$f(n) = g(n) + h(n),$$

where $g(n)$ is the cost accumulated from the start and $h(n)$ is the estimated remaining cost to the goal. In the cooperative version, the state includes position and time:

$$s = (r, c, t),$$

where r and c are grid coordinates and t is a time step. The planner checks candidate moves against obstacles and time-indexed reservations. Implemented conflict checks include vertex conflicts (two agents occupying the same cell at the same time) and edge-swap conflicts (two agents traversing the same edge in opposite directions during the same interval). A wait action can be considered when moving immediately is not feasible.

Cooperative A* plans agents in priority order. It can produce cost-aware routes under the search assumptions, but prioritized planning does not guarantee a globally optimal joint plan for all agents.

5.2 Greedy Best-First Search (GBFS)

GBFS prioritizes states using

$$f(n) = h(n).$$

It considers the estimated remaining cost but does not include the cost already incurred, $g(n)$, in its priority. It often follows states that appear closer to the goal and may therefore expand fewer states. However, it has no general guarantee of finding a shortest route.

5.3 Why GBFS Expanded Fewer Nodes in These Runs

In the recorded experiments, GBFS expanded fewer nodes in all four scenarios. Its heuristic-only priority directed the search toward states estimated to be closer to the goal, whereas A* also considered accumulated path cost and could explore more alternatives. This is an observation from these runs, not a universal rule: GBFS is not guaranteed to expand fewer nodes on every map.

6 System Workflow

1. Initialize the airport grid, obstacles, depot, gates, agents, and assignments.
2. Select one of the four scenarios and select Cooperative A* or GBFS.
3. Plan routes using the selected algorithm.
4. Check candidate movements against the implemented reservation and conflict rules.
5. Execute routes; perform service and return movements where applicable.
6. Replan when a predefined event changes the planning conditions.
7. Collect performance metrics and compare the algorithms.

7 Scenarios and Edge Cases

- **Normal Operation:** Agents travel to their assigned gates under the normal configuration.
- **Gate Unavailable:** Gate B becomes unavailable. In this scenario, the affected agent is redirected to Gate D and its route is replanned.
- **Emergency Priority:** At simulation time step 5, FUEL01 receives emergency priority and the fleet is replanned using the updated priority order.
- **Unreachable Goal:** A destination is isolated by obstacles. The system reports that no path is available for the affected agent rather than treating the task as completed.

These scenarios test normal route planning, event-driven replanning, priority changes, and failure to find a feasible path. No corridor-specific scenario or corridor-specific reservation feature is claimed in this report.

8 Complexity Analysis

Let b be the branching factor, d the solution depth, and N the number of states in the finite search graph.

For general search trees, both A* and GBFS can have exponential worst-case time and space complexity, commonly expressed as

$$O(b^d).$$

This is a general bound and is not a claim that every run explores that many states.

For a finite grid with a time-expanded state representation, a rough upper bound on the number of possible states is

$$N \leq R \times C \times T,$$

where R and C are the grid dimensions and T is the number of time steps considered. Obstacles and unreachable states reduce the actual number of states.

With a binary-heap priority queue, bounded-degree transitions, and standard duplicate handling, a typical graph-search time bound is approximately

$$O(N \log N).$$

The logarithmic factor arises from priority-queue insertion and removal. This bound assumes each state generates a bounded number of transitions and does not undergo an unbounded number of reinsertions. Both algorithms can have this same finite-graph bound; they may differ in how many states they actually expand.

The search memory for the finite state space is typically

$$O(N),$$

for the frontier, explored-state information, costs, and route reconstruction. The reservation table requires additional memory proportional to the number of stored reservation entries, denoted Q, so total memory also includes $O(Q)$. Exact bounds depend on the implementation, state horizon, duplicate handling, and reservation representation.

9 Experimental Results

The following values are the recorded results from the project's saved metrics file. Runtime is measured in milliseconds.

Scenario	Algorithm	Dist.	Wait	Makespan	Conflicts	Nodes	Runtime (ms)	Done / No pa
Normal	A*	84	1	28	0	154	1.1180	4
Normal	GBFS	84	1	28	0	93	1.2379	4
Gate unavailable	A*	68	1	29	0	126	1.0339	3
Gate unavailable	GBFS	68	1	29	0	77	1.1859	3
Emergency	A*	94	1	34	0	389	1.9756	4
Emergency	GBFS	96	1	36	0	348	1.8397	4
Unreachable goal	A*	18	0	21	0	32	0.3477	1
Unreachable goal	GBFS	18	0	21	0	20	0.3477	1

Table 1: Recorded scenario results for Cooperative A* and GBFS.

9.1 Interpretation

- **Normal Operation:** Both algorithms recorded the same distance and makespan. GBFS expanded 93 nodes compared with A*'s 154.
- **Gate Unavailable:** Both algorithms recorded distance 68 and makespan 29. GBFS expanded 77 nodes compared with A*'s 126.
- **Emergency Priority:** A* recorded distance 94 and makespan 34; GBFS recorded distance 96 and makespan 36. A* expanded 389 nodes, while GBFS expanded 348. In this run, A* used more search expansions but produced better aggregate route metrics.
- **Unreachable Goal:** Both algorithms recorded one completed vehicle and one vehicle with no path. The distance and makespan describe the recorded scenario outcome and do not mean that the unreachable vehicle reached its goal.

GBFS expanded fewer nodes in all four recorded runs, but this did not always translate into lower runtime. Runtime measurements are very small and can be affected by per-state work, priority-queue operations, reservation checks, implementation overhead, and measurement variation. These recorded results should not be interpreted as universal guarantees. Repeated runs and verified timing instrumentation would strengthen runtime conclusions.

10 Conclusion

The case study demonstrates grid-based multi-agent route planning under normal operation, gate unavailability, emergency priority, and unreachable-goal conditions. The environment is modeled as fully observable, deterministic, sequential, dynamic, discrete, known, and multi-agent.

In the recorded experiments, GBFS expanded fewer nodes in all four scenarios. Cooperative A* achieved the same aggregate distance and makespan as GBFS in the first two scenarios and better recorded distance and makespan in the emergency scenario. The results illustrate a trade-off between search effort and route quality; they do not establish that one algorithm is universally superior.

Further evaluation with larger grids, more agents, repeated dynamic events, repeated timing trials, and additional metrics would help establish how the algorithms behave as the planning problem becomes more complex.